

Lecture 7 Graph(I), Graph(II)

Graph Terminology

Graph: $G(V, E)$ consists of a set of V of vertices and a set of E of linking pairs of vertices. These pairs of vertices are called edges.

Undirected graph: each edge is undirected. $((v_i, v_j) = (v_j, v_i))$

Directed graph: each edge is directed. $(\langle v_i, v_j \rangle \neq \langle v_j, v_i \rangle)$

Complete Graph: a graph that has **maximum number** of edges. (for directed graph, $n(n-1)$ edges of n vertices; for undirected graph, $n(n-1)/2$ edges of n vertices)

Weighted graph: Each edge of the graph has a weight.

Subgraph: $G = (V, E)$, $G' = (V', E')$, if $V' \in V$, $E' \in E$, G' is the subgraph of G .

Simple graph: without self-loop or parallel edges.

Path from v_i to v_j : a sequence of vertices such that each pair of vertices in it have an edge.

Length of a path: the number of edges on the path.

Simple path: the vertices in the path are distinct (Not include initial and final vertices).

Cycle: a simple path that the first and the last vertices are the same.

Acycle: a graph without cycles.

Directed acycle graph(DAG): a directed graph without cycles.

Two vertices are connected: There is a path between the vertices.

An undirected graph is connected: For every pair of distinct vertices, there is a path.

Tree: a graph that is **connected and acyclic**.

Connected component(undirected graph): a **maximal** connected subgraph.

Strongly connected(directed graph): for every pair of two vertices, there is a path from a vertex to another and vice versa.

Weakly connected(directed graph): Not strongly connected but connected without direction.

Strong connected component: a maximal subgraph that is strongly connected.

Degree(v): The number of edges incident to the vertex v . For directed graph, it has in-degree(v as the head) and out-degree(v as the tail). And we have

$$e = \frac{\left(\sum_{i=0}^{n-1} degree\right)}{2}$$

Graph Representation

Adjacent Matrix

Represented by a two-dimensional matrix.

The vertices are indexed from 0 to $n-1$, let $A[n][n]$ be a $n \times n$ matrix.

For unweighted graph:

$$A[i][j] = \begin{cases} 1, & \text{if } (v_i, v_j) / < v_i, v_j > \text{ is in the graph} \\ 0, & \text{otherwise} \end{cases}$$

All elements on the diagonal are 0 for simple graph.

The matrix is **symmetric** for **undirected graph** and $\text{degree}(v_i) = \sum_{j=0}^{n-1} A[i][j]$.

For **directed graph**, row is in-degree and column is out-degree.

For weighted graph:

$$A[i][j] = \begin{cases} w_{ij}, & \text{if } (v_i, v_j) / < v_i, v_j > \in E, w_{ij} \text{ is the weight of the edge} \\ \infty, & \text{otherwise} \end{cases}$$

Space complexity is $O(n^2)$.

Adjacent List

An array of linked list. The i-th linked list stores the vertices that are adjacent to vertex v_i .

For directed graph, there are adjacent list, in-degree and adjacent list, out-degree.

For convenience, the number of the degree is stored in the head of each list.

Space complexity is $O(v + e)$.

Graph Traverse

Depth First Search (DFS)

First visit a neighbor, then proceed to traverse all the unvisited neighbors of the neighbor in similar fashion. Then proceed to traverse the remaining neighbors of A in similar fashion

The **tree edge** (u, v) is the edge that first from the visited vertex u to unvisited vertex v . And the **DFS tree** is the connected component of all tree edges. Every vertices have a **back edge** to it visited neighbors(exists in graph but not in tree).

Breath First Search (BFS)

First visit k neighbors, then traverse all the unvisited immediate neighbors of each neighbor until the graph is completely traversed.

If the edges taken during the breadth-first traversal are marked, then they define a tree. Such a spanning tree is called **BFS tree**.

Topological Ordering (拓扑排序)

Suppose G is a **directed graph without cycles**. The topological ordering is **a sequential listing of the vertices** such that for any pair of vertices, v and w in G , if there is a path from v to w in G , then v precedes w in the sequential listing.

It can check whether a graph is acyclic.

Algorithm 1:

- Calculate the in-degree of each vertex and save them in an array `indegree[|V|]`
- Find all vertices whose in-degrees are 0 and output them.(If no, it is cyclic)
- Decrease the indegree of the vertices adjacent to the output vertices
- Repeat the step 2 and 3 until all vertices are output.

Time complexity: $O(|V|^2)$.

It is not necessary to traverse all vertices. We just need to check the neighbors of the vertices whose in-degrees are 0.

Algorithm 2:

- Counting the in-degree of each vertex;
- All vertices with in-degree 0 are enqueue;
- If the queue is empty, go to 7;
- Remove the first vertex N from queue and output it;
- The in-degrees of all the neighbors adjacent to N is decremented by 1. enqueue vertices whose in degrees become to 0;(If no, it is cyclic)
- Go to 3;
- If all vertices have been output, the printed order is topological ordering. Otherwise, the graph include cycles.

Time Complexity: $O(|V| + E)$

Topological Ordering by DFS

- Starting from the first vertex, do DFS. Probe from the first neighbor. Mark the vertices on the path as visited, and output the vertex which has no unmarked successors.
- Next the recursion backs to the first vertex, and probe other neighbors in similar fashions until it has no successors. Then output it.
- Then pick the next unmarked vertex, continue the process until all vertices are output. Then we can obtain a reversed topological ordering. (Note that there is usually more than one topological ordering for a graph.)